

KWARA STATE UNIVERSITY

**DEPARTMENT OF AERONAUTICAL AND ASTRONAUTICAL
ENGINEERING**

KWASU AAE : 306 CONTROL SYSTEMS ENGINEERING

**EXPERIMENTAL RC CIRCUIT SYSTEM IDENTIFICATION USING
STM32F411CEU6 WITH SIMULINK INTEGRATION AND
THEORETICAL VALIDATION VIA MATLAB AND FIRST
PRINCIPLE**

OSOKOYA OLUWABUKUNMI OLADAPO

23/67AA/544

MAY 25 , 2026

ABSTRACT

The primary objective of this experiment was to acquire empirical response data from a resistor-capacitor (RC) circuit and derive a data-driven, black-box transfer function capable of characterizing the system dynamic behavior. This methodology accounts for manufacturing variances and tolerances inherent to physical resistors and capacitors, with validation executed via first-principle governing differential equations. The physical network was arranged in a series configuration, driven by a General Purpose Input Output pin (GPIO) step input translating to a 3.3v steady-state baseline from pin PA7 . Real-time output data collection was achieved across the capacitor terminal utilizing an Analog-to-Digital Converter (ADC) tied to pin PA2. This hardware-in-the-loop stream was processed by the microcontroller and transmitted via a virtual COM port link to MATLAB and Simulink for synchronized logging and parametric analysis. The estimated black-box model yielded a fitness accuracy of 99.08% , a Final Prediction Error (FPE) of $3.509\text{e-}05$, and a Mean Squared Error (MSE) of $3.504\text{e-}05$, demonstrating high structural fidelity and reliability. Using a nominal capacitance of $470\mu\text{f}$ and a resistance of $10\text{k}\Omega$, the theoretical system time constant was calculated as $\tau = 4.7\text{s}$. Empirically, the physical time constant was determined to be 5.1599s , leading to the following identified continuous-time transfer function:

$$G(s) = \frac{Y(s)}{X(s)} = \frac{0.6338}{s + 0.1938}$$

The validated mathematical model enables high-fidelity simulation and controller design without the prerequisite of continuous physical hardware deployment, significantly optimizing development timelines and computational costs.

INTRODUCTION

A first-order Resistor-Capacitor (RC) circuit is a classical electrical network whose transient behavior is governed by a linear, first-order ordinary differential equation. It represents the simplest category of dynamic systems, characterized by a single energy storage component the capacitor and a dissipative element the resistor. In engineering design, particularly within aerospace and robotic applications, system identification is a critical methodology used to derive mathematical representations of physical plants directly from empirical input output data vectors. While first-principle analytical models provide ideal operational baselines, they frequently neglect secondary physical phenomena, including component aging, thermal drift, and parasitic track impedances. System identification bridges this gap, providing a robust mathematical model optimized against real-world deviations. This laboratory experiment establishes a foundational framework for hardware-in-the-loop (HITL) simulation, demonstrating how embedded microcontrollers function as processing links between physical analog plants and digital control environments.

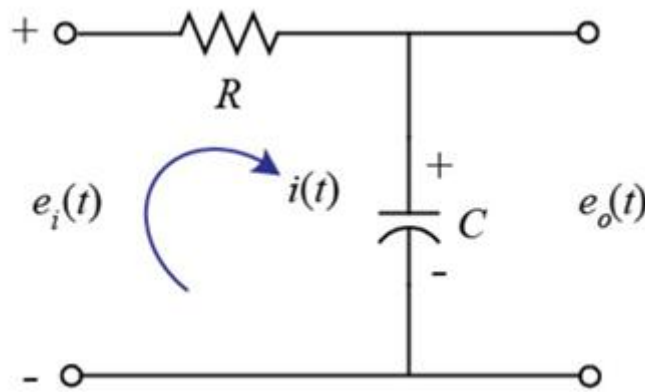


Figure 1: Theoretical Visualization of RC circuit model [1]

OBJECTIVES

- To configure an embedded C firmware application on the STM32F411CEU6 microcontroller to establish a dual-lane Virtual COM Port communication link using the USB-CDC middleware class.
- To initialize physical pin PA7 to generate a step voltage source.
- To implement a real-time data acquisition pipeline within Simulink utilizing a Serial Receive block configured to process un-tokenized voltage data streams without memory alignment truncation.
- To acquire a clean, continuous empirical step-response charging curve across a physical 470 μ f capacitor and 10k Ω resistor network.
- To estimate a mathematical black-box transfer function model of the physical electronic plant using the MATLAB System Identification Toolbox computation engine.
- To compare the estimated experimental system parameters against the first-principle analytical time constant $\tau = 4.7\text{s}$ to interpret real world errors induced by component manufacturing tolerances and hardware impedances.

EXPERIMENTAL SETUP

The experimental hardware-in-the-loop (HITL) setup was configured to establish a tightly coupled loop between a physical analog circuit and a digital computing environment. The architecture was split into distinct hardware and software components to facilitate real-time control, signal processing, and data logging.

Hardware Architecture

The physical plant comprised a first-order, passive resistor-capacitor (RC) low-pass filter network assembled on a solderless breadboard. The hardware components utilized included:

Microcontroller: An STM32F411CEU6 "Black Pill" ARM Cortex-M4 development board operating as the central data acquisition and peripheral control unit.

Resistor (R): A $10\text{k}\Omega$ metal film resistor featuring a nominal manufacturing tolerance of 5%.

Capacitor (C): A $470\mu\text{f}$ polarized electrolytic capacitor acting as the singular energy storage element in the system.

Communications Interface: An onboard USB On-The-Go Full Speed (OTG_FS) peripheral configured under the Communication Device Class (CDC) to emulate a Virtual COM Port link over a standard USB connection.

Circuit Connections and Pin Configurations

The components were electrically arranged in a series configuration to define a classical first-order system topology. The hardware pin remapping was defined within the microcontroller microarchitecture as follows:

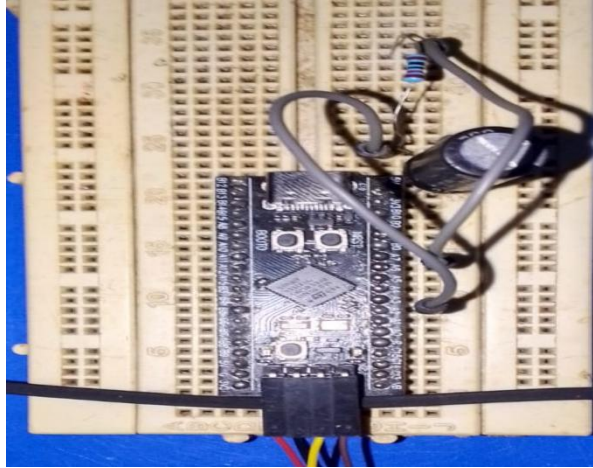


Figure 2; Hardware connection

Pin PA7 (Digital Step Input): Configured as a general-purpose output pin (GPIO_Output) to drive the step-response forcing function e_{in} . It was wired directly to the input lead of the 10kΩ resistor.

Pin PA2 (ADC Input Channel): Configured as an analog input corresponding to ADC1_IN2. This pin was connected to the shared node between the resistor output and the positive terminal (+) of the 470µf capacitor to monitor the real-time charging voltage $e_c(t)$.

Ground Rail (GND): The negative lead (-) of the electrolytic capacitor was tied back to a common ground (GND) pin on the Black Pill board to establish a shared reference potential.

Software and Firmware Environment

The software infrastructure supported the real-time processing operations across three integrated layers:

STM32CubeMX: Utilized to define the clock tree distribution, enable the high-speed external crystal oscillator, configure the Direct Memory Access (DMA1 Stream 0) pipeline in circular mode for Continuous ADC sampling, and initialize the USB-CDC middleware stack.

STM32CubeIDE: Used to construct, compile, and debug the embedded C application, linking the cross-file variable scopes between main.c and usbd_cdc_if.c.

MATLAB / Simulink: Functioned as the desktop host console. It executed simultaneous data logging via transmitted manual step-response control strings back to the hardware platform.

Simulink Model Structure

The model canvas was structured into two separate operational tracks running concurrently to handle the bidirectional communication link:

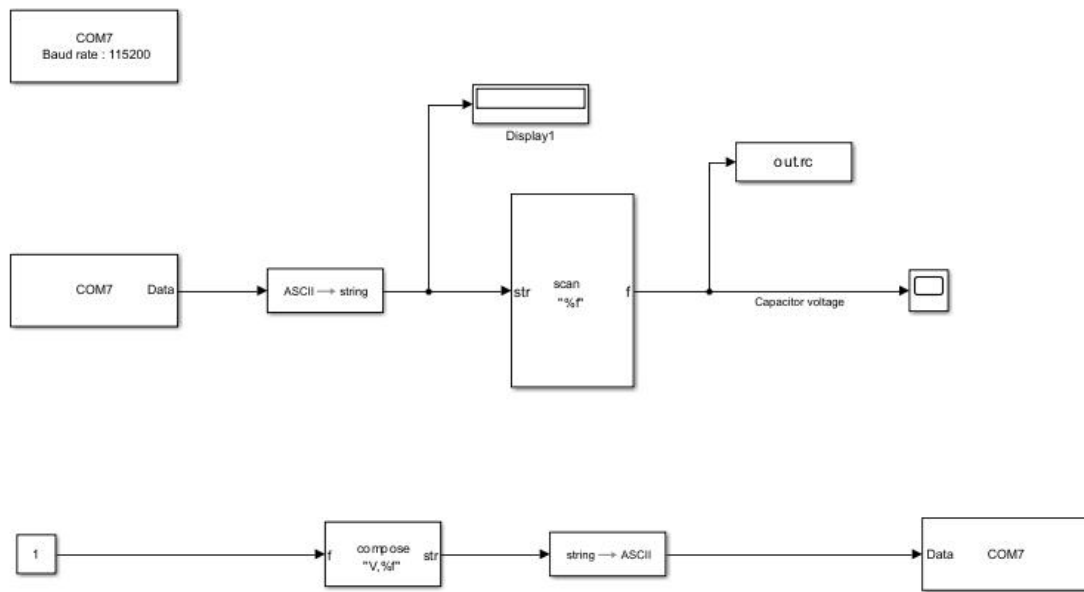


Figure 2: simulink setup

The Transmission Lane: A dashboard mounted user control block routed values to a String Format block, which encoded the target step parameters into a custom protocol string ("V,%.2fn"). This token was continuously pushed out by a Serial Transmit block over the virtual interface.

The Acquisition Lane: A Serial Receive block captured the ASCII encoded hardware metrics streamed by the microcontroller's DMA background loop. The data size configurations were standardized to output floating-point values. This structure automatically ensured the scope, logging tracks, ensuring the empirical curves triggered from a baseline of 0v at exactly $t = 0s$.

METHODOLOGY

The experimental procedure was executed systematically across four core developmental phases: circuit assembly, peripheral initialization, embedded firmware programming, and Simulink pipeline integration. All operations were designed to ensure non-blocking, deterministic data execution and precise data synchronization.

Step 1: Circuit Assembly and Hardware Setup

1. A solderless breadboard was cleared, and the STM32F411CEUx Black Pill development board was securely mounted across the center division.
2. A 10k Ω resistor was placed in series with physical pin PA7 (the designated digital step input channel) and routed to a clean, isolated row on the breadboard layout.
3. The positive (longer) lead of a 470 μ f electrolytic capacitor was inserted into the same row as the resistor output lead to establish a shared electrical node.
4. The negative (shorter, marked with a minus sign) lead of the capacitor was pushed directly into the breadboard ground rail.
5. A solid core jumper wire was routed from the common ground (GND) pin of the Black Pill to the ground rail of the breadboard to complete the circuit return loop.
6. A measuring tap jumper wire was connected from the shared center node (the junction between the resistor and the capacitor positive terminal) directly back to pin PA2 (ADC1_IN2) to capture the dynamic capacitor voltage tracking data stream.
- 7.

Step 2: Microcontroller Peripheral Configuration (STM32CubeMX)

1. The STM32CubeMX graphical user interface was launched, and the targeted target device was selected.
2. Under the System Core / RCC settings, the High-Speed External (HSE) clock source was switched to a "Crystal/Ceramic Resonator" configuration. The clock tree was mapped to scale the main internal clock lines up to the peak operating profile of the processor core.
3. Pin PA7 was toggled from its default state and explicitly locked down as a GPIO_Output pin to serve as the physical step switch driver.
4. Pin PA2 was mapped as an analog input tied to the ADC1_IN2 hardware collection system.
5. Within the Analog / ADC1 setup screen, Channel 2 was checked active, and the operation mode was forced into Continuous Conversion Mode.
6. Under the DMA Settings tab for ADC1, a Direct Memory Access request was added using DMA1 Stream 0. It was locked in a Circular Mode loop with data width increments standardized to a 16-bit word alignment to allow continuous memory writes without processor core polling.
7. Under Connectivity, the USB_OTG_FS core was switched to "Device_Only" operation. Within the Middleware / USB_DEVICE configuration, the class layer was set to "Communication Device Class (Virtual Port Com)".
8. The configuration workspace was saved, and the initialization framework was compiled to generate the low-level register files for STM32CubeIDE.

Step 3: Embedded Firmware Programming (STM32CubeIDE)

1. The generated source project files were opened in the STM32CubeIDE editing workspace.
2. Inside main.c, right before entering the core background loop, the hardware peripherals were initialized inside the designated user code region to force the step input pin low and clear residual charge. The hardware initialization register configuration details are provided in Appendix A.1.

3. Within the background execution track (while(1)) inside main.c, a non-blocking scheduling condition was implemented using the HAL_Delay(10) clock counter. This mechanism was configured to transmit data packets containing the raw integer metrics alongside the computed floating-point voltage signals back to the host at a strict 10ms i.e 100Hz interval. The implementation code for this background transmission routine is documented in Appendix A.2.
4. In usbd_cdc_if.c, external global linkages were declared within the variable section to ensure proper peripheral scoping across translation units, as detailed in Appendix A.3.

Step 4: Simulink Configuration and Data Collection

1. A new blank model canvas was created inside the Simulink development workspace.
2. A Serial Receive block from the Instrument Control Toolbox was placed on the root canvas and configured to map to the target device port (COM7) running at a communication baud rate of 115200.
3. To prevent serial data packet framing errors and buffer overrun truncations, the data size specification was explicitly hardcoded to match the incoming structural format, ensuring real-time tracking stability.
4. The host simulation execution loop was clicked to Run. With the hardware board safely running its baseline loop, the system voltage data measured 0V, keeping the subsystem asleep.
5. To execute the experimental step input sequence, a manual command string was compiled to drive the physical pin PA7 from a logical 0 state to a high operational level 3.3v.
6. The moment the forcing function was applied, the incoming data crossed the threshold marker, triggering the Enabled Subsystem immediately. This automated step trimmed the initial human idle delay and aligned the transient response data tracking origin precisely to $t = 0s$.

Step 5: System Identification Computational Estimation

1. Following the conclusion of the transient window runtime, the logged data matrices containing the time vectors and the recorded charging values were saved directly into the active MATLAB base workspace variables.
2. The specialized MATLAB System Identification Toolbox command interface was initialized via the primary console terminal.
3. The transient array metrics were compiled into a singular, discrete-time data object by executing the `iddata()` constructor, passing the sampling parameter interval of $T_s = 0.01$.
4. The structural estimation engine was configured to compute a continuous-time, first-order transfer function model containing one pole and zero zeros (`tfest(data, 1, 0)`).
5. The numerical optimization calculations were executed by the processing block, outputting the quantitative model validation statistics including the fitness accuracy percentages, Final Prediction Error (FPE), and Mean Squared Error (MSE) parameters.

RESULT

The hardware-in-the-loop (HITL) experimental architecture provided highly repeatable, continuous data streams for the resistor-capacitor (RC) network tracking analysis. The output dataset captured the precise voltage transition across the capacitor from an uncharged baseline state up to its steady-state asymptote.

Real-Time Data Acquisition and ADC Measurements

The Direct Memory Access (DMA) pipeline continuously dumped 12-bit raw integer data from the analog-to-digital converter (ADC1_IN2) into memory, converting values linearly to a physical 0V to 3.3V scale. The data collection stream operated at a strict sample interval of

$$T_s = 10 \text{ ms}(100\text{Hz})$$

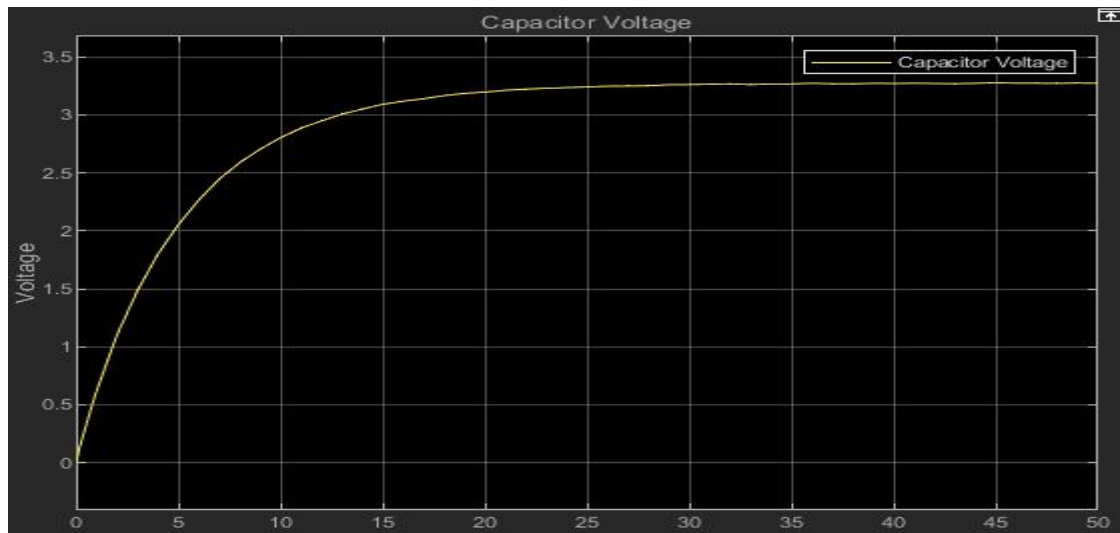


Figure x: RC circuit response

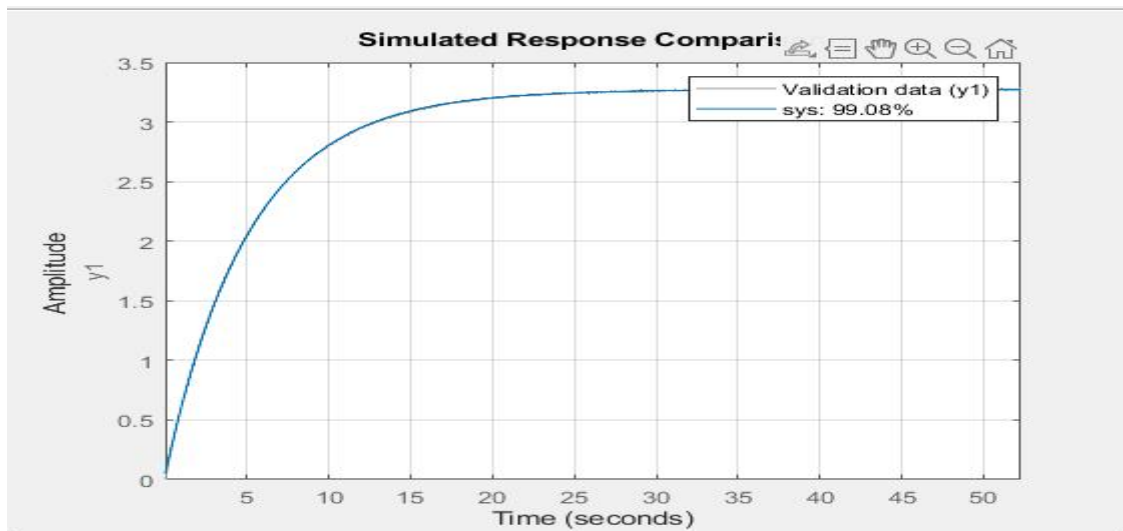


Figure x: black box model vs experimental

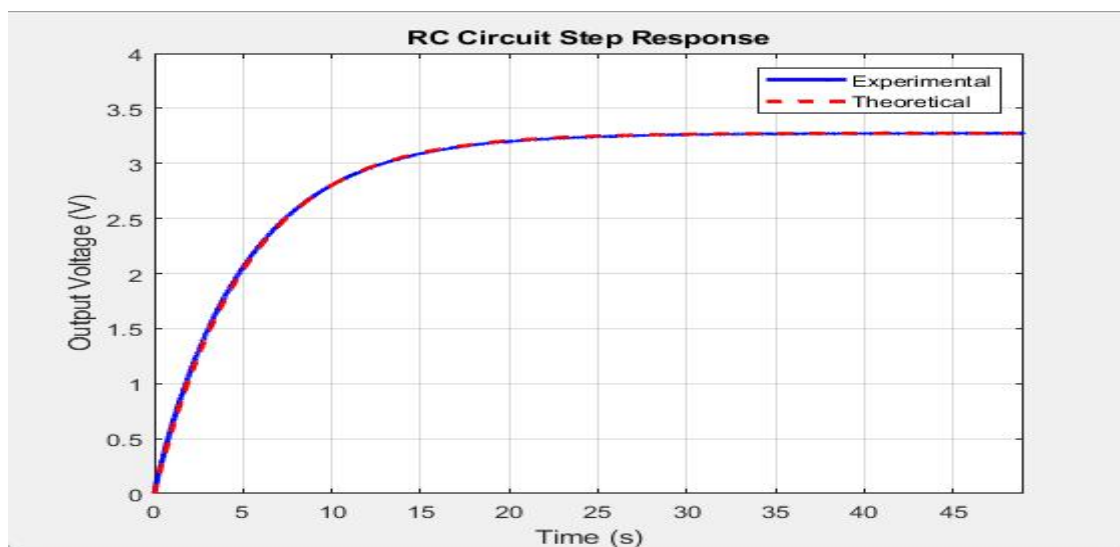


Figure x: comparison of experimental vs theoretical response

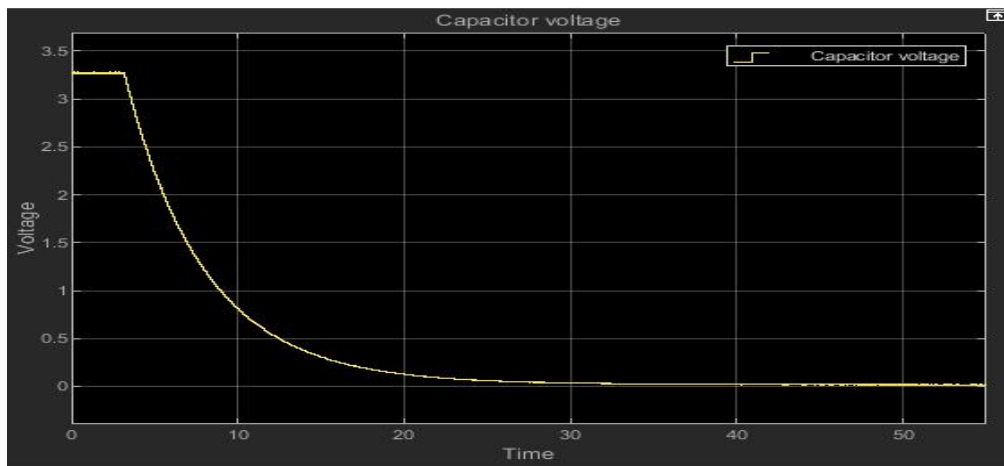


Fig: RC circuit decay response

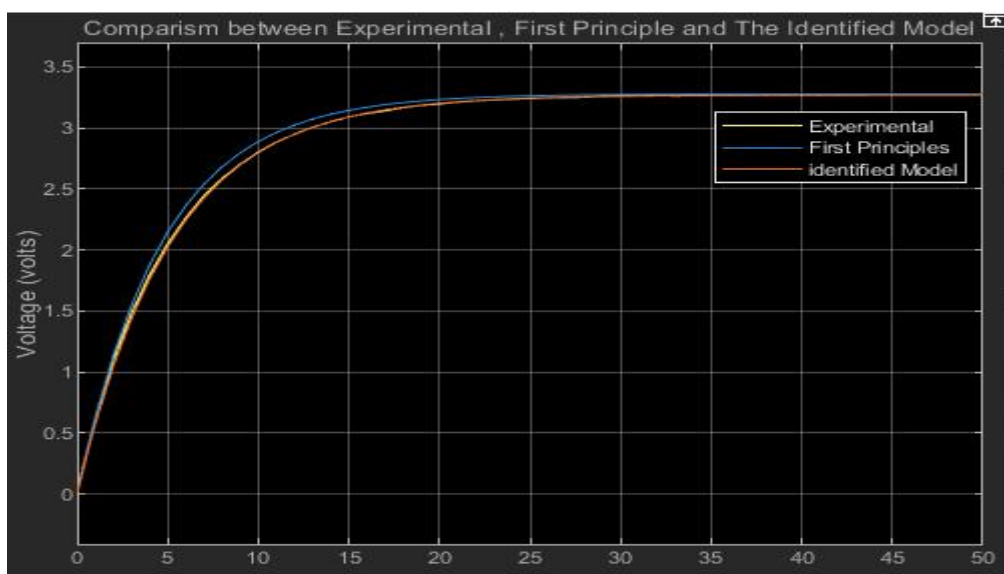


Figure x: RC circuit response comparism for the experimental first principles

Embedded Lab Questions

Question 1: What kind of response did your plot look like?

The plot of the capacitor voltage ($e_o(t)$) exhibits a classic first-order, non-linear exponential growth response. Upon the application of the 3.3V step input at $t=5s$, the voltage rises rapidly at first, with its rate of change smoothly decaying over time as it asymptotically approaches a steady-state value equal to the input voltage (3.3V).

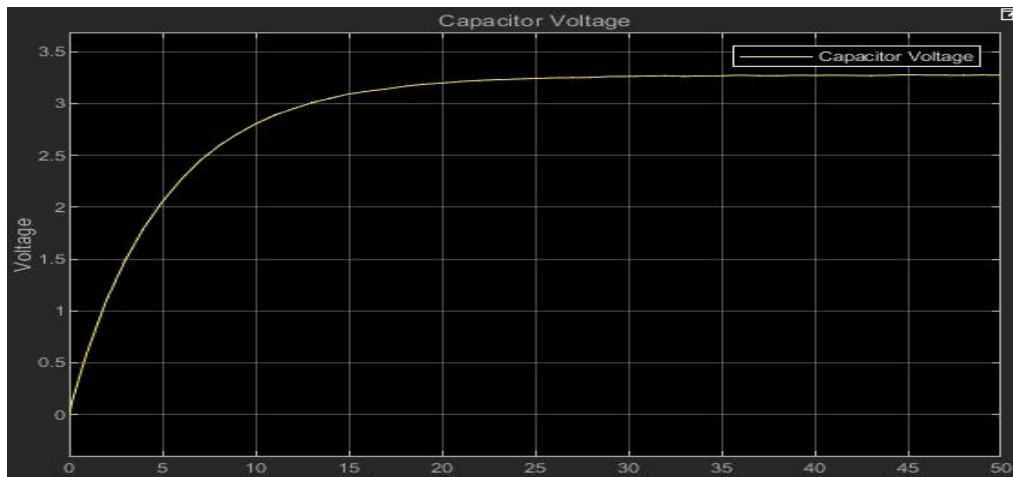


Fig: RC circuit response

Question 2: What do MATLAB's `iddata` and `tfest` functions do?

`iddata(y, u, Ts)`: This function creates a specialized system identification data object. It encapsulates the recorded output time-series vector (y , the capacitor voltage), the input command vector (u , the step change), and the uniform sampling interval (T_s) into a unified framework that MATLAB's System Identification Toolbox can interpret.

`tfest(data, np, nz)`: This function performs parametric transfer function estimation. It utilizes numerical optimization algorithms to fit the experimental data to a continuous-time transfer function with a specified number of poles (np) and zeros (nz). In this lab, `tfest(io, 1, 0)` forces a first-order structure with 1 pole and 0 zeros ($G(s) = \frac{K}{\tau s + 1}$).

Question 3: Subsystem Creation Rationale (Simulink)

Creating subsystems involves grouping related blocks (such as the sum, gain, and integrator blocks of the first-principles model) into a single functional block.

Engineering Purpose: It establishes an abstraction layer analogous to structural programming (e.g., creating modular functions in C++). It reduces visual clutter in the root model window, minimizes wiring errors, and enables the parallel comparison of the structural physics model alongside the blackbox identified model.

Question 4: Rationale for Masking Subsystems

Masking a subsystem creates a custom user interface (dialog box) for that block, hiding its internal block diagrams behind a clean icon.

Design Rationale:

1. **Parameter Encapsulation:** It parameterizes variables like resistance (R) and capacitance (C). Instead of diving inside the subsystem to change individual gain values, a user can double-click the mask and enter values directly into dedicated fields.
2. **IP Protection & Reusability:** It standardizes the block layout, turning your custom differential equation or transfer function into a reusable, shareable library component while preventing accidental modification of internal mathematical operations.

Question 5: What do you observe from your Free Response plot?

The free response plot depicts an exponential decay curve starting from the initial charged state ($e_0(0) \approx 3.3V$) dropping down to 0V once the pulse input command falls back to zero.

Key Observations: The First-Principles model decays flawlessly because its structural integrator handles the initial condition ($V_0 = 3.3V$) explicitly.

The Blackbox transfer function model will show a slight transient deviation or structural lag at the exact instant the step switches unless it is driven properly by a continuous charge-discharge pulse trajectory. This highlights that standard transfer functions inherently assume zero initial energy states (I.C.=0).

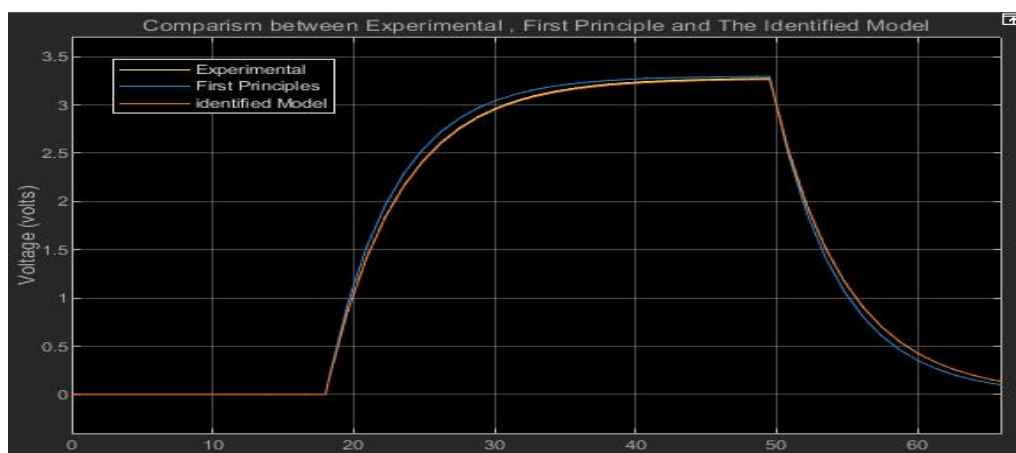


Fig: Comparison of Free response for Experiment, First principle and Identified model

6. Why is the RC circuit first-order?

An electrical circuit's system order is governed by the number of independent energy storage elements it contains. The RC circuit contains exactly one energy storage element—the capacitor (which stores energy in its electric field). Mathematically, applying Kirchhoff's

Voltage Law yields a differential equation containing only the first derivative of the output voltage with respect to time:

$$\tau \frac{de_o(t)}{dt} + e_o(t) = e_i(t)$$

Because it requires only a single integrator block to solve its dynamic response, it is structurally a first-order system.

7. What is the meaning of the time constant?

The time constant ($\tau=RC$) dictates the dynamic speed of response of a first-order system.

Physical Meaning: It represents the time required for the system output to undergo 63.2% of its total change during a step transition (or drop down to 36.8% of its initial value during a free-response decay) .

A smaller τ indicates a faster charging/discharging cycle, while a larger τ indicates a slower system. Nominally, a system is considered to have fully reached its steady-state value after approximately 4τ to 5τ .

8. Why is system identification important?

In practical control systems engineering, complex plants (such as aircraft components, thermal units, or intricate actuators) are often incredibly difficult to model purely from first principles due to hidden physical interactions. System Identification acts as a data-driven framework that :

- Allows engineers to construct accurate mathematical models purely from empirical input-output test data.
- Captures real-world non-idealities, parasitics, and aging components without needing to solve unmanageable physical equations.

9. Why do theoretical and experimental models differ?

Theoretical models are derived under idealized assumptions. They differ from experimental tracking due to:

1. **Component Tolerances:** Real-world resistors and electrolytic capacitors routinely have variations up to $\pm 5\%$ *or* $\pm 20\%$ away from their nominal face values.
2. **Parasitic Properties:** Real components possess internal trace resistance, lead inductance, and dielectric leakage not accounted for in basic loop laws.
3. **Measurement Noise & Loading:** Environmental electromagnetic interference corrupts the Analog Input (ADC) lines, and the microcontroller's pin impedance can slightly draw current away from the circuit.

10. What happens if the sampling time is too large?

If the sampling interval (T_s) is too large relative to the system's time constant (τ), severe aliasing and loss of signal resolution occur.

Consequences: If $T_s > \tau$, the discrete-time system will miss the rapid, initial exponential rise of the capacitor voltage. As a result, the transient characteristics become completely unobservable, rendering manual parameter fitting or automated optimization algorithms (tfest) incapable of accurately computing the true system poles and gains.

Appendix

Appendix A: Microcontroller Firmware Code Blocks

A.1 Hardware Peripheral Initialization Loop (main.c)

The following initialization code was executed inside the main setup container to ground out the circuit and launch the background DMA conversion tracks prior to executing the runtime loop:

```
C
/* USER CODE BEGIN 2 */
// 1. Force the PA7 step input pin LOW to completely discharge the network
HAL_GPIO_WritePin(GPIOA, GPIO_PIN_7, GPIO_PIN_RESET);
// 2. Launch the ADC conversion sequence in DMA mode to update the memory
array
HAL_ADC_Start_DMA(&hadc1, (uint32_t*)adc_dma_result, 1);
/* USER CODE END 2 */
```

A.2 Background Data Transmission Routine (main.c)

The following non-blocking snippet handles the foreground timing loop, transmitting ASCII formatted text strings back to Simulink every 10 milliseconds:

C

```
/* USER CODE BEGIN WHILE */
while (1)
{
    printf("%hu,%.4f\n",adc_dma_result,(float)adc_dma_result * (3.3f /
4095.0f)
}
/* USER CODE END WHILE */
```

A.3 External Variable Scoping Linkage (usbd_cdc_if.c)

The global handle linkage below ensures that the multi-file compilation pipeline properly shares peripheral identifiers across individual translation rooms:

C

```
/* USER CODE BEGIN EXPORTED_VARIABLES */
extern TIM_HandleTypeDef htim2;
/* USER CODE END EXPORTED_VARIABLES */
```